# RNA-seq Differential Expression Analysis using DESeq2

Oğuzhan Işılay

2026-03-07

## Contents

**Introduction** 1

**Dataset** 2

**Loading the RNA-seq Dataset** 2

**Extracting the Count Matrix and Sample Metadata** 3

**Creating a DESeq2 Dataset** 5

**Running the DESeq2 Analysis** 6

**Inspecting Differential Expression Results** 6

**MA Plot** 7

**Volcano Plot** 8

**Heatmap of Significant Genes** 9

**Pathway Enrichment Analysis** 11

**Conclusion** 11

**Reproducibility** 12

## Introduction

In this document, we will analyze RNA-seq data using the **DESeq2** package in R.

The goal of this tutorial is not only to run the code, but also to understand why each step is performed during the analysis.

In this workflow we will cover the following steps:

- loading RNA-seq data
- preparing the count matrix and sample information
- creating a DESeq2 dataset object
- running differential expression analysis
- exploring sample structure using PCA
- visualizing results using MA plot and volcano plot
- showing significant genes with a heatmap
- performing a simple pathway enrichment analysis

This tutorial is written mainly for students in biotechnology and bioinformatics who want to learn the basic workflow of RNA-seq analysis.

The explanations try to keep the analysis simple and clear so that beginners can follow the logic of the steps.

---

## Dataset

In this tutorial we will use the **airway** dataset.

The airway dataset is commonly used for teaching RNA-seq analysis with DESeq2.
It contains RNA-seq count data together with sample information.

The dataset includes two main components:

- **count matrix** – raw RNA-seq counts for genes
- **metadata (colData)** – information about the samples

The sample information allows us to compare different experimental conditions and perform differential expression analysis.

Using this dataset helps us focus on learning the analysis workflow without needing to preprocess raw sequencing files.

---

## Loading the RNA-seq Dataset

In this section we will load the RNA-seq dataset that will be used in the analysis.

The dataset used in this tutorial is called **airway**.
It is available through **Bioconductor**, which is a collection of R packages developed for bioinformatics and computational biology.

The airway dataset contains RNA-seq data from human airway smooth muscle cells.

It includes both:

- gene expression counts
- sample information describing the experimental conditions

To access this dataset we first need to load the required packages.

```r
#install.packages("BiocManager")

library(airway)

library(DESeq2)
```

After loading the packages, we can load the dataset into the R environment.

```r
data(airway)
```

The object called **airway** is a special data structure used in Bioconductor called a **SummarizedExperiment**.

This structure stores multiple types of information together, such as:

- the count matrix
- sample metadata
- feature annotations

To understand the structure of the dataset, we can inspect the object.

```r
airway
```

```
## class: RangedSummarizedExperiment
## dim: 63677 8
## metadata(1): ''
## assays(1): counts
## rownames(63677): ENSG00000000003 ENSG00000000005 ... ENSG00000273492
##   ENSG00000273493
## rowData names(10): gene_id gene_name ... seq_coord_system symbol
## colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521
## colData names(9): SampleName cell ... Sample BioSample
```

---

# Extracting the Count Matrix and Sample Metadata

In RNA-seq analysis, two types of information are required for most workflows.

The first one is the **count matrix**.

The count matrix contains the raw RNA-seq counts for each gene in each sample.

In this matrix:

- rows represent genes
- columns represent samples
- values represent the number of sequencing reads mapped to each gene

The second important component is the **sample metadata**.

Metadata describes the biological or experimental information about each sample. For example:

- treatment condition
- cell line
- experimental setup

In the airway dataset, both the count matrix and the metadata are stored inside the **SummarizedExperiment** object.

We can extract the count matrix using the `assay()` function.

```
counts <- assay(airway)
```

Now we can check the dimensions of the count matrix.

```
dim(counts)
```

```
## [1] 63677     8
```

This command shows the number of genes and the number of samples in the dataset.

To inspect the first few rows of the matrix, we can run:

```
head(counts)
```

```
##                 SRR1039508 SRR1039509 SRR1039512 SRR1039513 SRR1039516
## ENSG00000000003        679        448        873        408       1138
## ENSG00000000005          0          0          0          0          0
## ENSG00000000419        467        515        621        365        587
## ENSG00000000457        260        211        263        164        245
## ENSG00000000460         60         55         40         35         78
## ENSG00000000938          0          0          2          0          1
##                 SRR1039517 SRR1039520 SRR1039521
## ENSG00000000003       1047        770        572
## ENSG00000000005          0          0          0
## ENSG00000000419        799        417        508
## ENSG00000000457        331        233        229
## ENSG00000000460         63         76         60
## ENSG00000000938          0          0          0
```

Next, we extract the sample metadata using the `colData()` function.

```
metadata <- colData(airway)
```

We can inspect the metadata with:

```
metadata
```

```
## DataFrame with 8 rows and 9 columns
##            SampleName     cell      dex    albut        Run avgLength
##              <factor> <factor> <factor> <factor>   <factor> <integer>
## SRR1039508 GSM1275862  N61311     untrt    untrt SRR1039508       126
## SRR1039509 GSM1275863  N61311     trt      untrt SRR1039509       126
## SRR1039512 GSM1275866  N052611    untrt    untrt SRR1039512       126
```

```
## SRR1039513 GSM1275867  N052611    trt      untrt SRR1039513        87
## SRR1039516 GSM1275870  N080611    untrt    untrt SRR1039516       120
## SRR1039517 GSM1275871  N080611    trt      untrt SRR1039517       126
## SRR1039520 GSM1275874  N061011    untrt    untrt SRR1039520       101
## SRR1039521 GSM1275875  N061011    trt      untrt SRR1039521        98
##            Experiment    Sample    BioSample
##              <factor>  <factor>     <factor>
## SRR1039508  SRX384345 SRS508568 SAMN02422669
## SRR1039509  SRX384346 SRS508567 SAMN02422675
## SRR1039512  SRX384349 SRS508571 SAMN02422678
## SRR1039513  SRX384350 SRS508572 SAMN02422670
## SRR1039516  SRX384353 SRS508575 SAMN02422682
## SRR1039517  SRX384354 SRS508576 SAMN02422673
## SRR1039520  SRX384357 SRS508579 SAMN02422683
## SRR1039521  SRX384358 SRS508580 SAMN02422677
```

The metadata contains information about the samples, such as treatment condition and cell line.

---

## Creating a DESeq2 Dataset

Before running DESeq2, we need to convert our data into a specific object used by the DESeq2 package.

This object is called a **DESeqDataSet**.

It basically combines three things:

- the count matrix
- the sample metadata
- the experimental design

In the airway dataset, information about the samples is stored in the metadata.

For example, the variable **dex** shows whether a sample is treated or untreated.

Now we can create the DESeq2 dataset using the count matrix and the metadata.

```r
dds <- DESeqDataSetFromMatrix(
countData = counts,
colData = metadata,
design = ~ dex
)
```

Here:

- **countData** contains the raw RNA-seq counts
- **colData** contains the sample metadata
- **design** defines the variable used in the analysis

---

## Running the DESeq2 Analysis

After creating the DESeq2 dataset object, we can run the analysis.

This is done using the **DESeq()** function.

```
dds <- DESeq(dds)
```

This function performs several steps automatically:

- normalization of the count data
- estimation of gene dispersion
- fitting the statistical model

After this step, the data is ready for differential expression analysis and result extraction.

---

## Inspecting Differential Expression Results

After running the DESeq2 analysis, we can examine the differential expression results for each gene.

This can be done using the **results()** function.

```
res <- results(dds)
```

This function returns a table containing the statistical results for every gene.

Let's take a quick look at the result table.

```
head(res)
```

```
## log2 fold change (MLE): dex untrt vs trt
## Wald test p-value: dex untrt vs trt
## DataFrame with 6 rows and 6 columns
##                   baseMean log2FoldChange     lfcSE      stat    pvalue
##                  <numeric>      <numeric> <numeric> <numeric> <numeric>
## ENSG00000000003 708.602170      0.3788470  0.173141  2.188082 0.0286636
## ENSG00000000005   0.000000             NA        NA        NA        NA
## ENSG00000000419 520.297901     -0.2037604  0.100599 -2.025478 0.0428183
## ENSG00000000457 237.163037     -0.0340428  0.126279 -0.269584 0.7874802
## ENSG00000000460  57.932633      0.1171786  0.301237  0.388992 0.6972820
## ENSG00000000938   0.318098      1.7245505  3.493633  0.493627 0.6215698
##                      padj
##                 <numeric>
## ENSG00000000003  0.139308
## ENSG00000000005        NA
## ENSG00000000419  0.183359
## ENSG00000000457  0.930572
## ENSG00000000460  0.895441
## ENSG00000000938        NA
```

The table includes several important columns:

- **log2FoldChange** → shows how much the gene expression changes between conditions
- **pvalue** → p-value obtained from the statistical test
- **padj** → p-value adjusted for multiple testing

In RNA-seq analysis, genes with **padj < 0.05** are usually considered statistically significant.

We can count how many genes are significant using:

```
sum(res$padj < 0.05, na.rm = TRUE)
```

```
## [1] 2700
```

---

# MA Plot

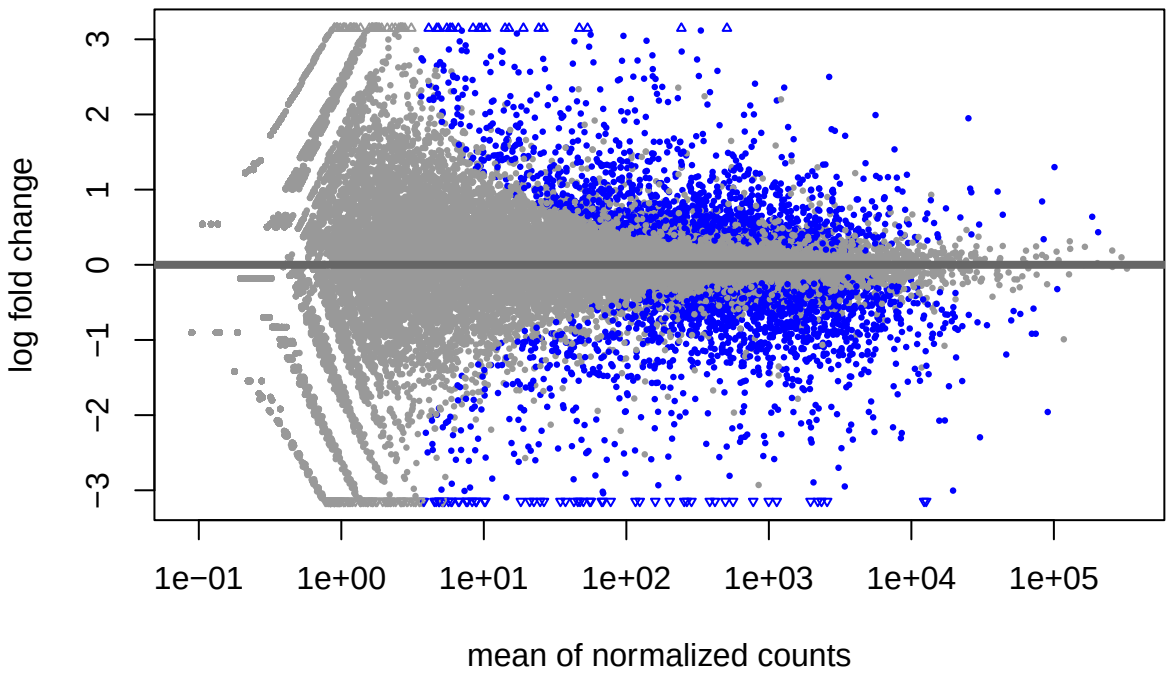
One of the easiest ways to visualize differential expression results is the **MA plot**.

An MA plot shows two pieces of information for each gene:

- the average expression level
- the expression change between conditions

In DESeq2, generating this plot is very simple.

```
plotMA(res)
```




In this plot, each point represents a gene.

In the graph:

- the **x-axis** shows the average expression level
- the **y-axis** shows the log2 fold change

Usually, the red points represent statistically significant genes.

This plot helps quickly identify genes that are strongly upregulated or downregulated.

---

## Volcano Plot

Another way to visualize differential expression results is by using a **volcano plot**.

A volcano plot shows two important values at the same time:

- gene expression change (log2FoldChange)
- statistical significance (-log10 p-value)

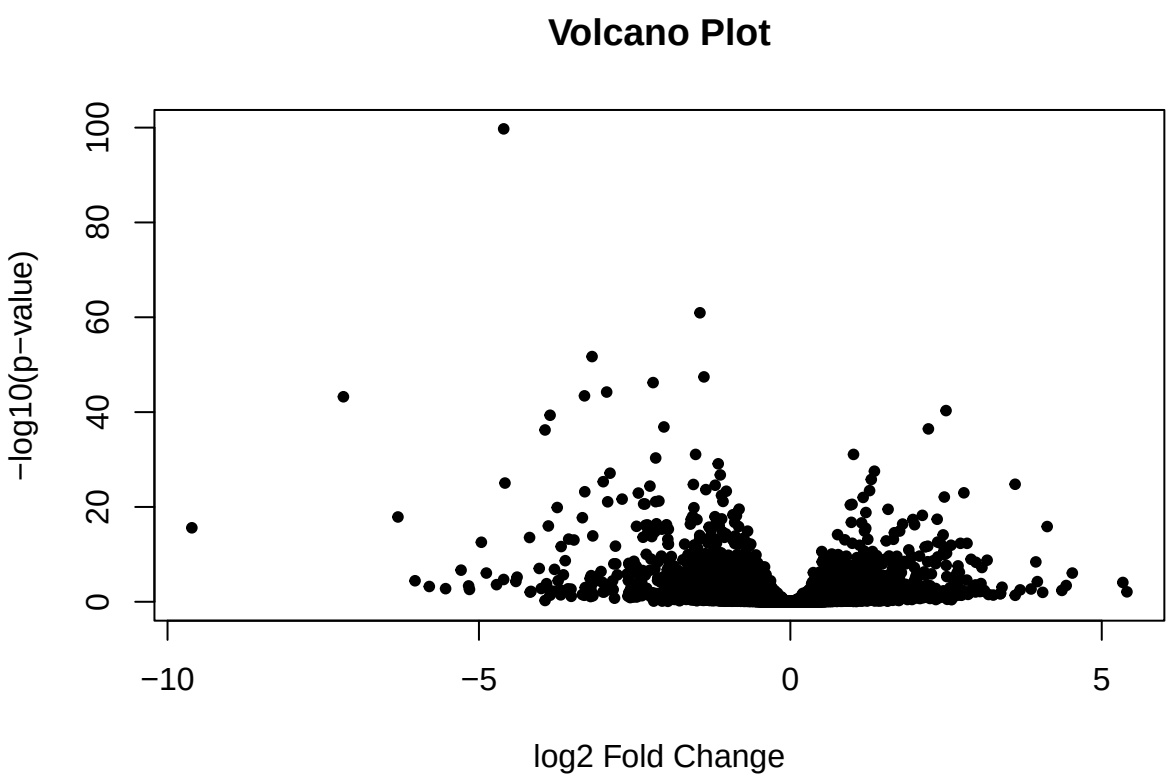
This allows us to quickly identify genes that show both strong expression changes and high statistical significance.

First, we convert the result table into a data frame.

```r
res_df <- as.data.frame(res)
```

Now we can generate the volcano plot.

```r
plot(res_df$log2FoldChange,
-log10(res_df$padj),
pch = 20,
main = "Volcano Plot",
xlab = "log2 Fold Change",
ylab = "-log10(p-value)")
```




In this plot:

- genes on the right side are **upregulated**
- genes on the left side are **downregulated**

The y-axis represents statistical significance.

This plot helps quickly identify genes that may be biologically interesting.

---

## Heatmap of Significant Genes

After differential expression analysis, it is often useful to examine the expression patterns of significant genes.

A common way to visualize these patterns is using a **heatmap**.

First, we select the significant genes.

Here we select genes with **padj < 0.05**.

```
sig_genes <- res[which(res$padj < 0.05), ]
```

Next, we extract the gene names.

```
sig_gene_names <- rownames(sig_genes)
```

Then we extract the expression values for these genes from the count matrix.

```
mat <- counts[sig_gene_names, ]
```

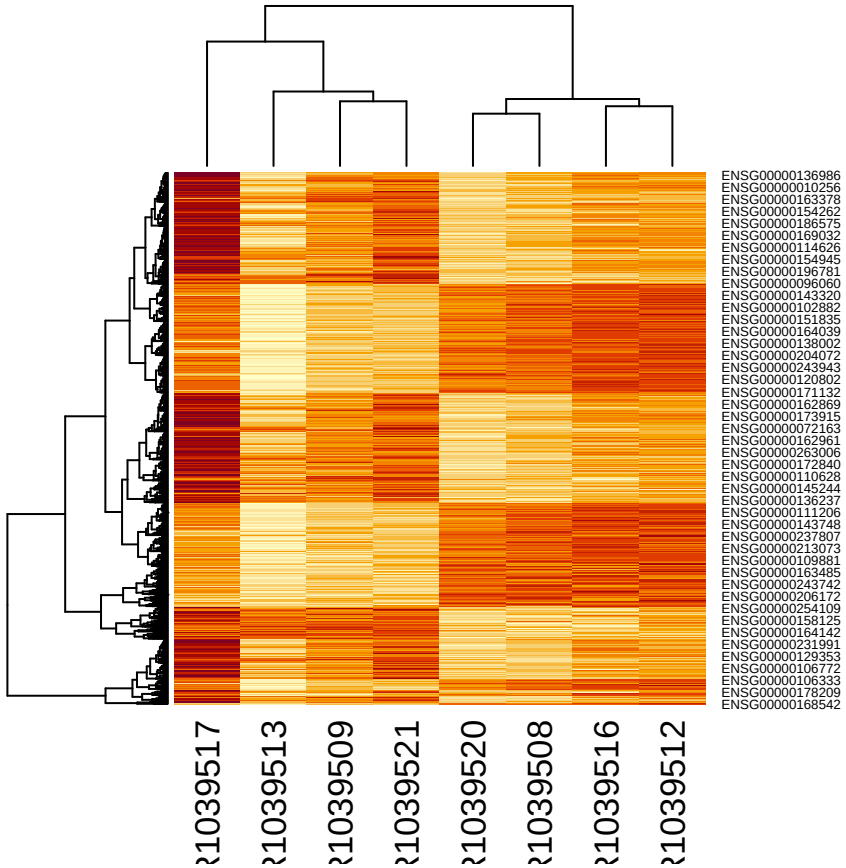
Before plotting the heatmap, it is often helpful to transform the data.

For example, we can apply a log transformation.

```
mat_log <- log2(mat + 1)
```

Finally, we generate the heatmap.

```
heatmap(mat_log)
```




This plot shows how significant genes are expressed across different samples.

Heatmaps help visualize patterns and similarities between genes and samples.

---

## Pathway Enrichment Analysis

After differential expression analysis, an important question is:

Which biological processes are associated with these genes?

To investigate this, we can perform **pathway enrichment analysis**.

This analysis tests whether the significant genes are overrepresented in known biological pathways.

First, we create a list of the significant gene names.

```
gene_list <- rownames(sig_genes)
```

Pathway analyses are usually performed using biological databases such as:

- KEGG
- GO (Gene Ontology)
- Reactome

In R, several packages can be used for enrichment analysis.
One commonly used package is **clusterProfiler**.

```
#BiocManager::install("clusterProfiler")

library(clusterProfiler)
```

Pathway enrichment analysis helps us understand the biological meaning of the genes identified in the differential expression analysis.

---

## Conclusion

In this tutorial, we demonstrated how RNA-seq data can be analyzed using the **DESeq2** package.

The workflow included the following steps:

- loading the RNA-seq dataset
- extracting the count matrix and sample metadata
- creating a DESeq2 dataset object
- running the differential expression analysis
- visualizing the results using different plots

These steps allow us to identify genes that show changes in expression between experimental conditions.

Visualization methods such as MA plots, volcano plots, and heatmaps help make the results easier to interpret.

This tutorial was written as a simple guide for students who are beginning to learn RNA-seq analysis with DESeq2.

---

## Reproducibility

For reproducibility, it is useful to record the R session information used during the analysis.

This includes:

- the R version
- operating system
- loaded packages and their versions

Recording this information helps ensure that the analysis can be reproduced in the future.

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Turkish_Türkiye.utf8  LC_CTYPE=Turkish_Türkiye.utf8
## [3] LC_MONETARY=Turkish_Türkiye.utf8 LC_NUMERIC=C
## [5] LC_TIME=Turkish_Türkiye.utf8
##
## time zone: Europe/Istanbul
## tzcode source: internal
##
## attached base packages:
## [1] stats4    stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] clusterProfiler_4.18.4      DESeq2_1.50.2
##  [3] airway_1.30.0               SummarizedExperiment_1.40.0
##  [5] Biobase_2.70.0              GenomicRanges_1.62.1
##  [7] Seqinfo_1.0.0               IRanges_2.44.0
##  [9] S4Vectors_0.48.0            BiocGenerics_0.56.0
## [11] generics_0.1.4              MatrixGenerics_1.22.0
## [13] matrixStats_1.5.0
##
## loaded via a namespace (and not attached):
##   [1] DBI_1.3.0               gson_0.1.0              rlang_1.1.7
##   [4] magrittr_2.0.4          DOSE_4.4.0              compiler_4.5.2
##   [7] RSQLite_2.4.6           systemfonts_1.3.1       png_0.1-8
##  [10] vctrs_0.7.1             reshape2_1.4.5          stringr_1.6.0
##  [13] pkgconfig_2.0.3         crayon_1.5.3            fastmap_1.2.0
##  [16] XVector_0.50.0          rmarkdown_2.30          enrichplot_1.30.5
##  [19] purrr_1.2.1             bit_4.6.0               xfun_0.55
##  [22] cachem_1.1.0            aplot_0.2.9             jsonlite_2.0.0
##  [25] blob_1.3.0              tidydr_0.0.6            DelayedArray_0.36.0
##  [28] tweenr_2.0.3            BiocParallel_1.44.0     cluster_2.1.8.1
```

```
##  [31] parallel_4.5.2          R6_2.6.1                stringi_1.8.7
##  [34] RColorBrewer_1.1-3      GOSemSim_2.36.0         Rcpp_1.1.1
##  [37] knitr_1.51              ggtangle_0.1.1          R.utils_2.13.0
##  [40] Matrix_1.7-4            splines_4.5.2           igraph_2.2.2
##  [43] tidyselect_1.2.1        qvalue_2.42.0           rstudioapi_0.18.0
##  [46] abind_1.4-8             yaml_2.3.12             codetools_0.2-20
##  [49] lattice_0.22-7          tibble_3.3.1            plyr_1.8.9
##  [52] withr_3.0.2             treeio_1.34.0           KEGGREST_1.50.0
##  [55] S7_0.2.1                evaluate_1.0.5          gridGraphics_0.5-1
##  [58] polyclip_1.10-7         scatterpie_0.2.6        Biostrings_2.78.0
##  [61] pillar_1.11.1           ggtree_4.0.4            ggfun_0.2.0
##  [64] ggplot2_4.0.2           scales_1.4.0            tidytree_0.4.7
##  [67] glue_1.8.0              gdtools_0.5.0           lazyeval_0.2.2
##  [70] tools_4.5.2             ggnewscale_0.5.2        data.table_1.18.2.1
##  [73] fgsea_1.36.2            locfit_1.5-9.12         ggiraph_0.9.6
##  [76] fs_1.6.6                fastmatch_1.1-8         cowplot_1.2.0
##  [79] grid_4.5.2              tidyr_1.3.2             ape_5.8-1
##  [82] AnnotationDbi_1.72.0    nlme_3.1-168            patchwork_1.3.2
##  [85] ggforce_0.5.0           cli_3.6.5               rappdirs_0.3.4
##  [88] fontBitstreamVera_0.1.1 S4Arrays_1.10.1         dplyr_1.2.0
##  [91] gtable_0.3.6            R.methodsS3_1.8.2       yulab.utils_0.2.4
##  [94] fontquiver_0.2.1        digest_0.6.39           SparseArray_1.10.8
##  [97] ggrepel_0.9.7           ggplotify_0.1.3         htmlwidgets_1.6.4
## [100] farver_2.1.2            memoise_2.0.1           htmltools_0.5.9
## [103] R.oo_1.27.1             lifecycle_1.0.5         httr_1.4.8
## [106] GO.db_3.22.0            fontLiberation_0.1.0    MASS_7.3-65
## [109] bit64_4.6.0-1
```